Now we need to set up our neural network model. It is a simple sequential (feedforward) neural network with two layers as we described earlier.

```
model = torch.nn.Sequential(
    torch.nn.Linear(D_in, H),
    torch.nn.ReLU(),
    torch.nn.Linear(H, D_out),
    torch.nn.ReLU(),
)
```

We'll use the mean squared error loss function here, but others could work too.

```
loss_fn = torch.nn.MSELoss()
```

Now we set up a new environment by instantiating the `ContextBandit` class, supplying the number of arms to its constructor. Remember, we've set up the environment such that the number of arms will be equal to the number of states.

```
env = ContextBandit(arms)
```

The main `for` loop of the algorithm will be very similar to our original *n*-armed bandit algorithm, but we have added the step of running a neural network and using the output to select an action. We'll define a function called `train` (shown in listing 2.10) that accepts the environment instance we created previously, the number of epochs we want to train for, and the learning rate.

In the function, we'll set a PyTorch variable for the current state, which we'll need to one-hot encode using the `one_hot(...)` encoding function:

```
def one_hot(N, pos, val=1):
    one_hot_vec = np.zeros(N)
    one_hot_vec[pos] = val
    return one_hot_vec
```

Once we enter the main training `for` loop, we'll run our neural network model with the randomly initialized current state vector. It will return a